Which Verification Strategy Makes Multi-Agent Financial Advisory Systems More Reliable?

Bhoomika Monthy Rajashekar Faisal Alsabih Tejas Vanjari Dhanya Kini Bailoor
Northeastern University

1 SUMMARY

LLM agents can draft portfolio rebalancing plans, liquidity plans, and policy-compliance reviews, but their answers read confidently even when the arithmetic or the constraint handling is wrong, and a client could act on the error. We take on the *verification* difficulty of Theme 6 (Track A, an agent with a verifier we build): drafting advice is easy, checking it is hard. We build a financial advisory pipeline and compare four conditions on the same synthetic tasks: no verification, self-verification, peer verification, and a dedicated verifier with a calculator tool. A deliberately weakened verifier and a cross-family verifier serve as ablations.

Business value and expected outcome. A wealth-management firm needs to know which control is worth its cost before deploying an advisory agent. We expect the dedicated, tool-equipped verifier to catch the most arithmetic and constraint errors and self-verification to help least, consistent with findings that verifiers help more when they differ from the solver [1] and that models struggle to self-correct without external feedback [2]. We expect the weakened verifier to erase part of the gain. We report each strategy's accuracy gain together with its token, dollar, and latency cost, so the result is a price per extra correct answer, not only an accuracy number.

2 SYSTEM ARCHITECTURE

Figure 1 shows the end-to-end data flow. Scenarios flow left to right into the advisor and the selected verification condition; every run is logged; an independent scorer, which uses evaluator-only files, evaluates the logged recommendations and feeds the metrics and report.

3 PROPOSED METHOD

Dataset. All data is synthetic, so no real client information is used. Ten pilot scenarios (Table 1) cover trade-plan rebalancing under restrictions, liquidity planning, policy-violation detection, a fee comparison, a retirement projection, rebalancing after a market change, and two deliberately infeasible requests. Each scenario is one JSON file holding the client profile, holdings and prices, investment policy, cash flows, a natural-language request, and approved execution assumptions (quoted prices, no fees or taxes, long-only, no borrowing or unlisted deposits). Answers take one of three types: trade plan, violation list, or numeric result. A generator will instantiate the templates into 30 to 40 tasks; feasible tasks are built backward from a known valid plan and infeasible ones are checked to be infeasible. The data model has 17 relational tables. Scoring by deterministic graders with explicit acceptance criteria follows prior wealth-management benchmarks [3].

Pipeline and conditions. Plain Python runs a fixed workflow with no router agent, so any difference between conditions comes from verification alone. One small, inexpensive model plays every role. We choose it with a pilot of about 20 tasks over candidates such as GPT-5.4 Nano, Gemini 2.5 Flash, Claude Haiku 4.5, and the open-weight GPT-OSS 20B, keeping the one whose baseline accuracy is near 50 to 80% so verification has errors to fix. Self-verification asks the advisor to re-check its own answer; peer verification runs a second, independent advisor call; the dedicated verifier has a checker prompt and a sandboxed Python calculator. Each verifier returns accept, reject, or abstain. A reject triggers one revision that is verified again; an abstain goes to a human review queue (Figure 2).

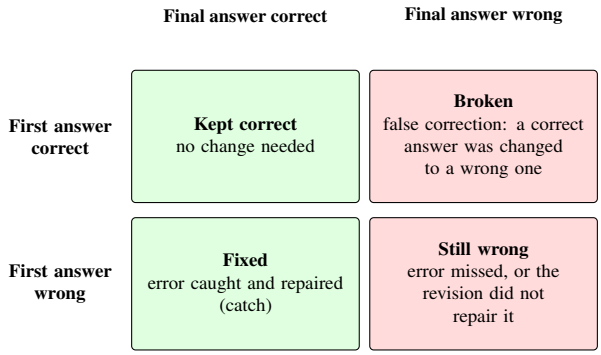


Figure 2: How each run is classified by comparing the first answer with the final answer.

Evaluation and budget. A deterministic scorer that agents never see replays the proposed trades at quoted prices, checks every restriction and allocation band, recomputes reported numbers, and accepts any valid solution rather than one reference plan. It scores both the first and the final recommendation. Metrics are task success; the verifier's verdict against whether the first answer was actually wrong (catch rate, false alarms, MCC); wrong answers fixed versus correct answers broken; and tokens, dollars, and latency, including extra cost per extra correct answer. Each task runs five times per condition, and we bootstrap over tasks and compare conditions with paired tests. Budget: 40 tasks $\times$ 6 conditions $\times$ 5 repeats $\times$ about 3 calls $\times$ about 2k tokens is at most about 7M tokens, within free-tier or low-cost budgets.

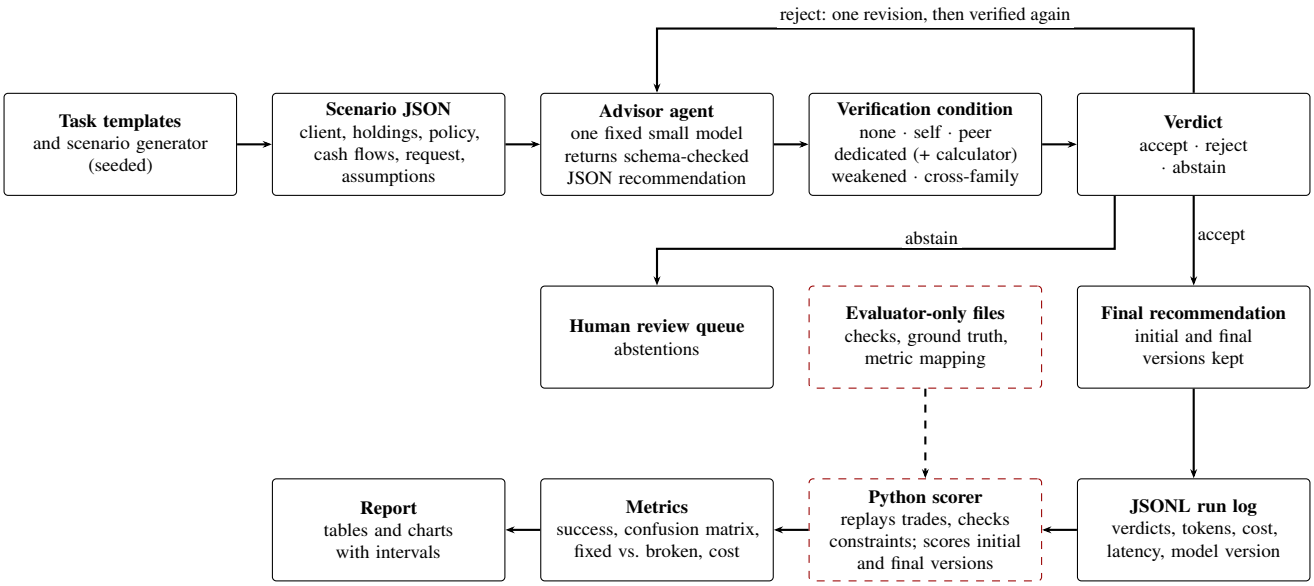


Figure 1: Architecture. Plain Python controls the workflow. Dashed red boxes are evaluator-only and are never given to any agent; every agent receives the complete scenario JSON and a role prompt.

Table 1: Pilot scenarios. Any valid answer passes; reference answers are not required.

ID	Task	Answer type	Expected status
S01	Rebalance to 60/30/10; AAPL no-sell, 20% stock cap, whole shares	Trade plan	Feasible
S02	Fund a $25,000 expense due in 90 days and keep at least $12,000 cash	Trade plan	Feasible
S03	Identify every violation of the client's investment policy	Violation list	Violations found
S04	Compare two funds' five-year values after all stated fees	Numeric result	Feasible
S05	Rebalance to 60/30/10; AAPL no-sell, 25% stock cap, 35% technology cap	Trade plan	Feasible
S06	Project the month-240 balance and the extra contribution needed to hit a target	Numeric result	Feasible
S07	Rebalance at new prices after a $12,000 deposit; AAPL no-sell, 25% stock cap	Trade plan	Feasible
S08	Withdraw $20,000, keep $10,000 cash, rebalance the remainder to 50/37.5/12.5	Trade plan	Feasible
S09	Rebalance to 40/50/10 without selling any stock-class asset	Trade plan	Infeasible
S10	Withdraw $80,000 and retain $5,000 cash, respecting all restrictions	Trade plan	Infeasible

Table 2: Metrics we report.

Metric	Definition
Task success	Every check passes on the final answer
Catch rate	Wrong first answers the verifier rejected
False-alarm rate	Correct first answers the verifier rejected
MCC	One summary of verdict vs. truth
Fixed vs. broken	Wrong→right vs. right→wrong after verification
Extra cost per extra correct answer	Added dollars ÷ added correct answers, vs. no verification

Challenges and ethics. The baseline may be too accurate or too weak for verification to matter, which the pilot is meant to prevent. Same-model self and peer checks may share blind spots, and small models may fail on output format before arithmetic. The scorer must itself be correct, so we unit-test it on hand-calculated cases, valid alternative solutions, and wrong outputs. With about 40 tasks, confidence intervals will be wide, and we will report them. To limit these risks we pilot each template first, fix numeric tolerances before any experiment, and pin model version strings in the run log. The main ethical risk is over-reliance on automated financial advice; we present outputs as decision support, let verifiers abstain to human review, and make no claims about real clients.

4 Related Work

A wealth-management benchmark finds that workflow reliability limits agents more than arithmetic does [3]. We differ by comparing self, peer, and tool-equipped verifiers in one financial pipeline with a constraint-based scorer, and by pricing each extra correct answer.

5 Team and Milestones

Faisal Alsabih	Dataset, data model, scenarios
Tejas Vanjari	Architecture and pipeline
Bhoomika Monthy Rajashekar	Models, tools, proposal
Dhanya Kini Bailoor	Scorer and evaluation
Nov 18	Progress report with baselines
Dec 9	Final report, slides, GitHub code

References. [1] J. Lu et al., "When Does Verification Pay Off?" arXiv:2512.02304, 2025. [2] J. Huang et al., "LLMs Cannot Self-Correct Reasoning Yet," ICLR 2024. [3] R. Milsom, "Benchmarking LLM Agents for Wealth-Management Workflows," arXiv:2512.02230, 2025.